

jiram_catalog

A pedagogical review: archive, geometry, and the two regimes

Companion to PEDAGOGICAL_REVIEW.md; every footer cites the review section it expands on.

Outline

What the tool is for

Three archive facts

Architecture

Deep dives

Figures from real data

Testing philosophy

How it was built

How to extend it

Glossary

Juno JIRAM, in one slide

- ▶ Two-channel infrared camera on Juno: L-band ($3.455\text{ }\mu\text{m}$, aurora, H3^+) and M-band ($4.780\text{ }\mu\text{m}$, thermal, day & night)
- ▶ Detector: $128\text{ lines} \times 432\text{ samples per band}$; 256-line LM products stack both bands, 10 gap lines removed
- ▶ Juno spins at 2 rpm (12 deg/s); JIRAM's despinning mirror holds one look steady per spin
- ▶ 85,108 archived camera frames as of this writing (README.md)

The archive's own labels do not tell you where a frame is pointed

They tell you which SPICE kernels were used to figure that out — computing it yourself is the first half of this project.

The single fact that organizes the whole codebase

A JIRAM frame is small; the spacecraft is fast. What the archive gives you looks completely different at the poles than everywhere else.

At the poles: Juno revisits
spin axis $\perp$ orbit plane $\Rightarrow$ JIRAM's one
pointing DOF sweeps the pole every pass; a
sequence returns every 8–20 min

Everywhere else: Juno never comes back
one swath, one look, gone — the spacecraft's
orbit carries it away for the rest of the mission

Regime 1: polar / repeat-view

- ▶ Real time-series data: same patch, reimaged, known cadence
- ▶ Natural product: a **region time stack** — every overlapping frame, reprojected onto one fixed grid, laid along a time axis
- ▶ Ground truth exists: Ingersoll et al. (2022, *Nature Astronomy*), 48 archived PJ4 frames → 4 published polar mosaics + wind vectors
- ▶ Built by: `region-stack`, `movie`, `export-goflow`

Regime 2: everywhere else

- ▶ One look, no revisit, no time axis worth having
- ▶ Natural product: a **library** — each look reprojected onto its own small tangent-plane grid, queryable by latitude / epoch / illumination
- ▶ No published ground truth; validated by internal consistency only
- ▶ Built by: `strips`, `strip-stats`

The two regimes, side by side

	regime 1: polar	regime 2: everywhere else
product	region time stack	strip library
grid	one per region	one per chunk
time axis	real, known cadence	none, one look
ground truth	Ingersoll et al. 2022	internal consistency only

When a module's shape is confusing: ask "is this regime 1 or regime 2?"

Three facts, undocumented, found the hard way

1. Archive labels lose all geometry after orbit 38
2. Image files are little-endian despite every label saying otherwise
3. Sequence numbering is 1-based, and frame 1 never arrives

Each one is enforced today by a specific line of code, not a comment. Get any of them wrong and geometry corrupts silently.

Fact 1: labels lose geometry after orbit 38

“label geometry is ABSENT for orbits ≥ 39 ... and for most 256-line dual-band products ... 64,139/85,108 frames have no label centre.”

—docs/build_log_2026-09-04.md, step 3d/3e

75% of the archive has no usable label geometry at all.

Fact 1: what it forced

- ▶ Label geometry could never be the foundation — only a spot-check
- ▶ `geometry.py + geo.py` → `frames_geo.parquet` became **mandatory** infrastructure, not a convenience
- ▶ Every downstream module reads `frames_with_geo()`; none fall back to label geometry, because for 3 in 4 frames there is nothing to fall back to

Fact 2: little-endian, despite every label

Every label declares IEEE754MSBSingle / IEEE_REAL — big-endian, unambiguously.

Read big-endian:

spans -2.6×10^{38} to 3.1×10^{38} , 143 NaNs

Read little-endian:

spans -0.00017 to $0.5547 \text{ W m}^{-2} \text{ sr}^{-1} \mu\text{m}^{-1}$
— matches the published map's 0.0060–0.5545

Found independently twice, from two unrelated questions, both times by checking whether the numbers were physically plausible.

Fact 2: what it forced

- ▶ Every raw-pixel read hard-codes '`<f4`' — never the label's declared type
- ▶ Now a project-wide rule (`AGENTS.md`), not a one-off patch
- ▶ The original crawl-index spec still says “big-endian” — explicitly superseded, left as historical record

Rule of this repo: where code and an old spec disagree, the code wins, and the spec stays as a record of what was believed at the time.

Fact 3: frame 1 never arrives

“sequence_number is 1-based and frame 1 is never delivered (seq_n = sequence_samples-1 always).”

—docs/build_log_2026-09-04.md, “Facts” after step 3d

Total, exact, unexplained — every sequence in the archive is missing its first frame.

Fact 3: what it forced

`sequence_number` cannot be trusted across a real gap either.

`index.assign_sequences` instead detects boundaries from observed behaviour:

new sequence starts when:

gap since previous frame > 45 s # *one spin ~30.5 s*
or `sequence_number` fails to increase

Every later grouping (strip chunking, trackability pairing) trusts this reconstruction, not the label's own claimed count.

Layers, in build order

1. `pds.py` + `mirror.py` — crawl and download the archive
2. `labels.py` + `index.py` — parse labels, build `frames.parquet`
3. `geometry.py` + `kernels.py` — the SPICE engine
4. `geo.py` — geometry for every frame → `frames_geo.parquet`
5. `vicar.py` + `reproject.py` — ground-truth I/O, map grids, inverse camera
6. `regions.py` → `stacks.py` / `strips.py` — the two regimes
7. `movie.py`, `export_goflow.py`, `stats2d.py`, `trackability.py` — exports and analysis
8. `tracking.py` — the standing acceptance test, not a pipeline stage
9. `api/` + `frontend/` — a view over everything above; computes nothing new

One frame's path: archive to product

```
PDS4 archive --(pds.py, mirror.py)--> mirrored .LBL/.IMG
  --(labels.py, index.py)--> frames.parquet (+ sequences)
  --(geometry.py, geo.py)--> frames_geo.parquet
  |
  +-- regime 1: select_frames -> reproject_frame -> region stack .nc
  | -> composite_sequences / movie.py / export_goflow.py
  +-- regime 2: unit_rows -> chunk_table -> build_strip -> strip .nc
  | -> stats2d.strip_statistics -> stats NetCDF
  |
  +-- GUI: reads every product above; writes only gui_cache/
```


Where every product lives on disk

product	path (relative to mirror root)
label index	index/frames.parquet
SPICE geometry	index/frames_geo.parquet
SPICE kernels	spice/{lsk,pck,fk,ik,sclk,spk,ck}/
region stacks	regions/<region>/<band>_orbits<spec>_<level>.nc
goflow realizations	regions/<region>/goflow_<band>_orbits<spec>/rNNNNN/
strip NetCDFs & index	strips/orbitNN/<strip_id>.nc, strips/strips.parquet
strip statistics	strips/stats/ or gui_cache/stats_<id>.nc
GUI cache/exports	gui_cache/, gui_cache/exports/

Deep dives: what each one covers

For every module: purpose, key data structures, the algorithm, what a modifier must know, the validation gate, and the scars.

17 modules, grouped by what they do:

- ▶ archive mirror & index (4.1–4.3)
- ▶ geometry & kernels (4.4–4.6)
- ▶ map machinery & regions (4.7–4.9)
- ▶ the two regimes (4.10–4.11, 4.13)
- ▶ validation & analysis (4.12, 4.14–4.15)
- ▶ the GUI, config (4.16–4.17)

Purpose: Apache listings → manifest → verified download.

- ▶ Only the Wget user agent is permitted under /PDS/; every download is a real wget subprocess
- ▶ Completeness = exact declared byte count from the sibling label, or last line END, or trailing </Product_Observational>
- ▶ -verify re-checks MD5 and re-downloads mismatches once

Scar: orbit50's listing alone is ~36 MB HTML — large enough to need retry-with-backoff, not a single timeout.

Purpose: one .LBL $\rightarrow$ one flat, typed dict — tolerantly.

- ▶ `pvl.load`, then a fallback grammar, then require the last line to be literally `END`
- ▶ A parse failure never raises: the row still exists, `parse_ok=False`, every geometry column `NaN`
- ▶ `"N/A"` (any case) is the *only* missing-value sentinel the archive uses
- ▶ `geom_band`: whichever band group has a numeric `CENTER_LATITUDE` — independent of which band the image data actually is

Purpose: multiprocessing label parse → frames.parquet, then segment sequences over the *whole* sorted table.

- ▶ orbit is float64 (4,697 nulls); **always use** orbit_dir, never orbit
- ▶ orbit disagrees with orbit_dir on 3,900 rows — all orbit03, labelled ORBIT_NUMBER=1; an archive property, not a bug
- ▶ sequence columns are recomputed over the combined table every run, never patched incrementally

Purpose: per pixel, per frame $\rightarrow$ lat/lon/range/emission/incidence/phase, no per-pixel Python loop.

```
x = ifov * (lines/2 + 0.5 - line) # 1-based line index
y = ifov * (samples/2 + 0.5 - sample) # 1-based sample index
d = (x, y, 1) / |(x, y, 1)| # unit ray, band frame
```

One vectorised numpy op over the whole (lines, samples, 3) grid — never per pixel. Then pxform: band frame $\rightarrow$ J2000 $\rightarrow$ IAU_JUPITER.

geometry.py: the aberration fix

sincpt's `srfvec` and every pixel ray are *apparent* (light-time + stellar aberration). Both must be un-aberrated *the same way* before the ellipsoid intercept.

Original spec: un-aberrate rays only

per-pixel error vs. oracle: **4e-3 deg**

Corrected: un-aberrate `srfvec` too

per-pixel error vs. oracle: **8e-5 deg** — a 50× reduction

Order of the mistake: $|v|/c \approx 7 \text{ arcsec} \approx 4 \text{ km}$ on Jupiter's surface at perijove range.

- ▶ Juno spins at 12 deg/s: a 0.1 s epoch error moves the boresight > 1 deg
- ▶ Epoch is always the label's START_TIME exactly — never mid-exposure (tested: off by *tens of degrees*)
- ▶ Engine is deliberately **mirror-blind**: models bus attitude only, not JIRAM's despinning mirror

Validated before writing code: mirror-blind SPICE matched 48 label centres to median 0.006 deg lat/lon, 0.017 deg emission, 28 km altitude.

- ▶ Static set (leap-seconds, planetary constants, one frame/instrument kernel) + per-orbit CK/SPK named in the labels
- ▶ Operational server first, PDS SPICE archive as fallback (renames SPKs: `spk_rec_*` $\leftrightarrow$ `juno_rec_*`)
- ▶ `naif.jpl.nasa.gov` fails system-CA TLS; every fetch pinned to the certifi bundle

Scar: 6 CK gaps >7 days found archive-wide; orbits 38/55/70 fail geometry from short gaps *not* among those six — still unidentified.

geo.py: augmenting every frame

- ▶ spawn workers (never fork) — SPICE kernel pool is per-process global state
- ▶ One row per (frame, band half); a 256-line LM product → 2 rows
- ▶ A frame's computation failure never aborts the run: `geo_ok=False`, error text recorded, row still exists

`bore_lat` $\neq$ label `CENTER_LATITUDE` for a partial frame

`bore_lat`: the boresight's own intercept, NaN if the boresight misses.

Label centre: a *visible-area centroid* for a partial (limb) frame.

Comparing the two without filtering to fully-on-planet frames looks like a large geometric error that is really just two different definitions of “centre.” **Open:** L-band residual (median 0.024 deg) is roughly double M-band's (0.010 deg) — both inside the M-only gate tolerance; cause unresolved.

- ▶ JPL's older format: ASCII label *inside* the file, not a sidecar
- ▶ Label size (LBLSIZE) always read from the file, never hard-coded — the paper's own supplementary readers hard-code 12,800 bytes and silently misparse the 1,728-byte .tp4 vector tables
- ▶ EOL=1: a *second* trailing label after the data (TRACKER4's own processing history)

reproject.py: the inverse camera model

`project_to_pixels`: run `geometry.py`'s chain backwards.

```
(lat,lon) -> surface point -> IAU_JUPITER -> J2000  
  -> re-apply FORWARD stellar aberration (put it back!)  
  -> band frame -> invert pinhole -> fractional (line, sample)
```

Round-trip residual on a live frame: **1.22e-05 px**.

`reproject_frame`: walk the *output* grid, pull from the source — no output pixel unpainted, no input pixel double-scattered.

reproject.py: stereographic was wrong

candidate	centroid RMS (px)	sing. ratio	km/px
stereographic	0.828	1.00178	15.069
orthographic (ellipsoid)	0.339	1.00010	15.005

Every JIRAM paper checked calls the polar maps “stereographic.” The fit says otherwise, decisively: the map *is* the body-fixed equatorial plane, orthographically projected — no trig at all once you have the 3-D point.

Three candidate radii (polar, equatorial, $\sqrt{ac}$) were proposed for the fit.

They cannot be distinguished by residual at all

$\rho \propto R$ for every radial law — registration only ever measures the *ratio* $R/\text{km_per_px}$, never R alone.

Settled instead by requiring self-consistency with the one independently stated number: the label's own $\text{MPS}=15.0$.

regions.py: two grid rules

- ▶ `polar_ortho`: the published rule, generalised to either pole, any central meridian — `north_pole_paper` reproduces `PAPER_GRID` exactly
- ▶ `local_ortho`: tangent plane at an arbitrary centre, dropped onto the ellipsoid along the local normal — every mid-latitude strip's grid

A new named region (e.g. the Great Red Spot) is a **4-line YAML addition**, no code.

regions.py: the visibility subtlety

Naive test $(P - c) \cdot up > 0$ fails: the tangent plane touches the ellipsoid *only* at c , so this is ≤ 0 almost everywhere — near side and far side alike.

Correct test: use the surface normal at P itself

$visible = (\text{normal}(P) \cdot up) > -tol$

Exactly the silhouette condition a rendering engineer would recognise.

stacks.py: selection, three cost tiers

`select_frames`, cheapest test first:

1. Scalar quality cuts on `frames_geo.parquet` — free
2. Bounding-box prefilter against the region — free, approximate
3. Exact test: fresh geometry, every 4th pixel, must land on canvas — expensive, exact

Three tiers, three cost levels, cheapest first — run the expensive exact test only on what already survived the free filters.

- ▶ One geometry-only pass unions every frame's footprint box *before* any pixel is reprojected
- ▶ Output array allocated **once**, at the tight window's size — never at the full region canvas (a north_pole region is 3600×3600)
- ▶ spawn workers each return only their own frame's footprint box, not the shared window

Orbit-4 frame stack: 294 steps, 2.25 GB. Sequence composite: 25 steps, more valid coverage than any one frame.

- ▶ **One whole-stack** brightness stretch, not per-frame — a per-frame stretch would *hide* the wobble that reveals a bad reprojection
- ▶ `export_goflow`: cuts variable-cadence stacks into constant-cadence *runs*; invalid pixels **zero-encoded** (not NaN) to match the downstream model's own convention
- ▶ No `u_mid/v_mid`: real data has no ground-truth velocity, unlike the synthetic sets this layout was designed to match

90% of a published map is exact-zero background; validity has to be a first-class concept, and the mask *depends on trial displacement*. 6 masked window sums $\times$ 1225 trial displacements per template — one at a time is hopeless.

Cost volumes

Each sum, at one fixed displacement, is a box filter over the whole tile — serves every template in that tile at once.

Measured throughput: **2,141 templates/s** on one core.

tracking.py: the reversed line axis

line axis	centre-on-footprint	full-template-on-footprint
direct	0.0%	0.0%
reversed	96.4%	91.4%

$\text{row} = \text{NL} - 1 - (\text{line} - 1)$ — settled unambiguously by footprint overlap.
0-vs-1-based index origin: genuinely underdetermined — the two conventions differ by 0.0005 px in the pooled result, “the tracking cannot separate them, which is the point.”

tracking.py: the gate numbers

	n	median (px)	frac < 1px
A: paper maps vs. TRACKER4	516,517	0.224	83.4%
B: our reprojections vs. TRACKER4	515,377	0.434	80.1%

The A→B gap (~ 0.2 px) is the honest cost of the full geometry+reprojection chain, on top of the tracker's own noise floor. **Scar:** 2 of 24 pairs (n0204aa/bb) sit near 1.04 px median — pooled statistic absorbs it, per-pair table reveals it.

strips.py: chunking a sequence

A single commanded sequence can slew tens of degrees while it runs. A chunk ends when:

- ▶ great-circle distance from the *chunk's first row* exceeds 12 deg, or
- ▶ the time gap exceeds 120 s

Chunks under `min_frames` are dropped but still consume a `chunk_index` — so a strip's name never shifts if the threshold changes later.

strips.py: resolution classes

RESOLUTION_CLASSES = (2, 3, 5, 7, 10, 15, 20, 30, 50, 70, 100, 150, 200, 300) km/px — log-nearest snap, ties to the finer class.

Why quantise at all

Two strips of the same class compare spectrum-to-spectrum, structure-function-to-structure-function, **with no resampling** — resampling would contaminate exactly the small-scale statistics being measured.

Result on the mirror: 289 strips, orbits 4+24, both bands, 392 MB.

Every diagnostic in this module follows one recipe, because a strip is a field with holes:

1. detrend over *valid* pixels only
2. fill invalid pixels with that same mean
3. taper the whole canvas (Tukey/Hann)
4. divide by $\text{valid_frac} \times \text{window_power}$

Result: a fully-valid, untapered field obeys Parseval exactly; a masked field is unbiased in *total* variance to first order.

Masking *convolves* the true spectrum with the mask's own spectrum — moves variance across k even when the total is right.

- ▶ Smooth, sparse holes: mild distortion. Measured: slope -2.97 unmasked vs. -2.92 with 15% masked
- ▶ Speckle masks: broadband leakage — **“no scalar correction can repair”** this (module's own words)

A documented, *measured* limitation, not hidden and not over-corrected.

$$b^2 = \frac{|\sum_s X_s(k_1)X_s(k_2)X_s^*(k_1+k_2)|^2}{\sum_s |X_s(k_1)X_s(k_2)|^2 \cdot \sum_s |X_s(k_1+k_2)|^2} \in [0, 1]$$

1.0 when the sum-mode phase is *locked* to the two parents' phases every segment; falls toward $1/n_{\text{segments}}$ when random. Test: same power spectrum, coupled vs.

independent third-term phase $\rightarrow b^2 > 0.5$ vs. $b^2 < 0.1$ at the same bin — detects coupling an ordinary spectrum cannot see at all.

trackability.py: before any pixel is reprojected

Pure numpy/pandas, no SPICE, no imagery. Candidate pair: same orbit/band, 90 s–6 h apart, boresights within a generous separation proxy.

The threshold alone over-admits

Dense polar mosaics: one boresight falls near *several* raster positions in the next sequence, not just its true repeat — verified numerically (closest wrong-position pair $<$ farthest correct-position pair).

Fix: keep only the **nearest partner per sequence** — on top of the threshold, not instead of it.

trackability.py: the best-baseline redefinition

Original: partner whose displacement is closest to 5 px

Revised: smallest dt among *trackable* partners

A sequence hours away could *coincidentally* land closer to 5 px than the true nearest repeat and win. “Smallest usable dt ” has no such failure mode. Totals: 29,862 unit frames, 16,541 with a partner, 7,493 trackable @ 30 m/s, 61 orbits.

api/ + frontend/: one store, typed arrays

- ▶ One zustand store shared by Catalog, Poles, Strips — so the tabs cannot disagree with each other
- ▶ Computes **no science**: every number shown is something the command line already wrote or could write
- ▶ Catalog (`/api/catalog/frames.arrow`, ~47,600 rows) split client-side into typed arrays; a filter is one pass producing a `Uint32Array` of surviving indices

- ▶ Server sends grayscale PNG, value in R, validity in alpha; browser maps it through a 256-entry lookup table (5 colour maps) — changing it is a redraw of pixels already in memory
- ▶ OrthographicView: zoom is a *scalar*, so aspect ratio is locked by construction, not by convention
- ▶ Catalog points drawn as a ScatterplotLayer, never rasterised — hover is real GPU picking over $\sim 47,600$ points

api/ + frontend/: the selection tray & jobs

- ▶ Selection tray: a Set of `product_id|half` keys, not row indices — survives a catalog that grows; feeds “Build stack...” and “Show in Strips”
- ▶ Long jobs (movie render, stack build, goflow export) run on a two-worker thread pool, polled from `/api/jobs` every 2 s
- ▶ Two gates: `test_gate_api.py` against the backend directly, `test_gate_frontend.py` running Playwright end to end against a live server

config.py + the subcommand pattern

- ▶ Precedence: explicit arg > env var > TOML file > built-in default — resolved *at call time*, never cached at import
- ▶ `add_subparser(subparsers) / run(args)`: a module owns its own CLI surface; `cli.py` wires it in afterward
- ▶ Enforces a project rule: executors never edit `cli.py`, `pyproject.toml`, `uv.lock` directly — exactly where concurrent milestones would collide

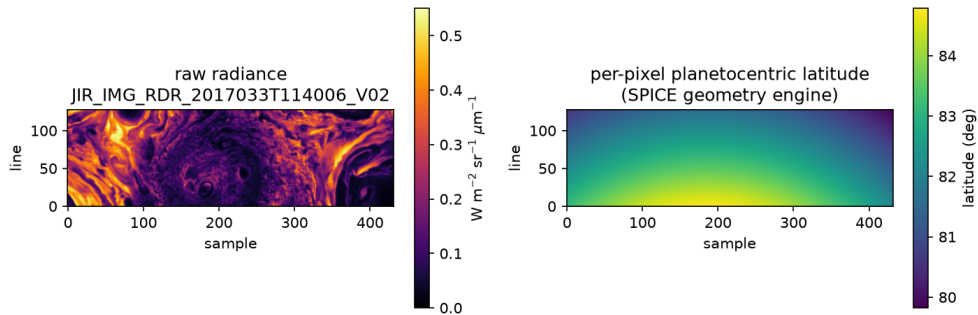
Six figures, all from real data on the mirror

Generated by `docs/pedagogy/make_figures.py`, reusing `jiram_catalog` directly
— no module modified.

- ▶ (a) one raw frame beside its computed latitude map
- ▶ (b) the published n01a map vs. our reprojection, with the difference
- ▶ (c) one mid-latitude strip with its graticule
- ▶ (d) that strip's isotropic spectrum and structure function
- ▶ (e) the trackability heatmap
- ▶ (f) a sketch of the SPICE frame chain and the pinhole model

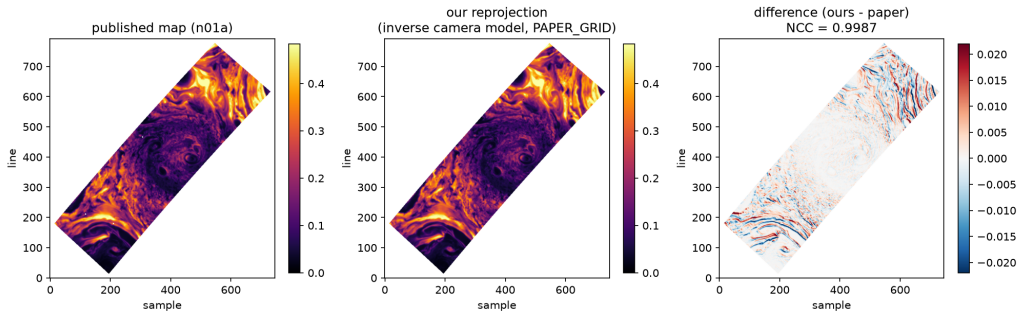
(a) A raw frame and its computed geometry

Figure (a): one JIRAM camera frame and its computed geometry



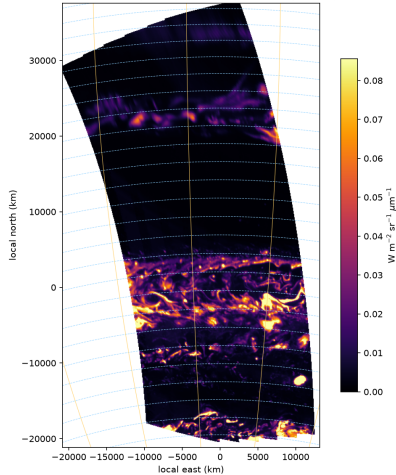
(b) Reproducing a published map from raw bytes

Figure (b): reproducing one published perijove-4 map from the raw archive frame



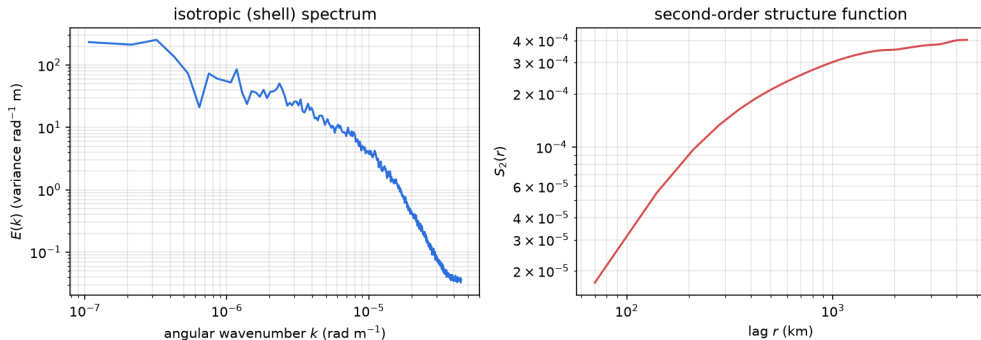
(c) A mid-latitude strip, with its graticule

Figure (c): strip 24_M_2019360T202519_01
orbit 24, band M, 70 km/px, centre -35.7 N, 343.6 E
(blue: latitude every 2 deg; gold: longitude every 10 deg)

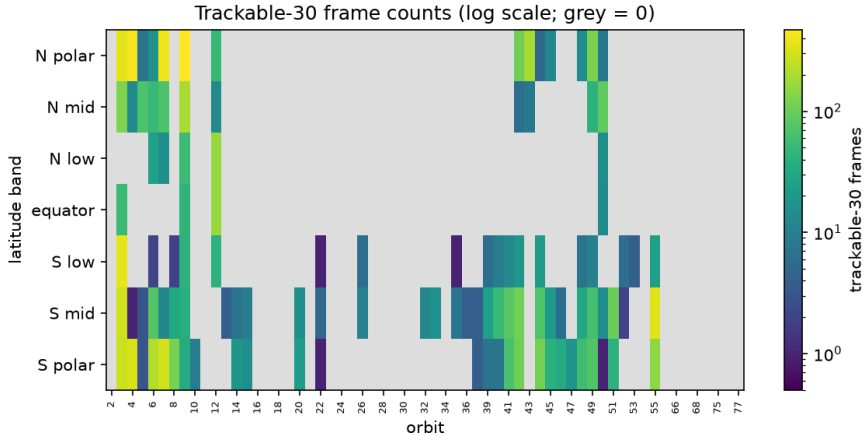


(d) That strip's spectrum and structure function

Figure (d): masked statistics of strip 24_M_2019360T202519_01

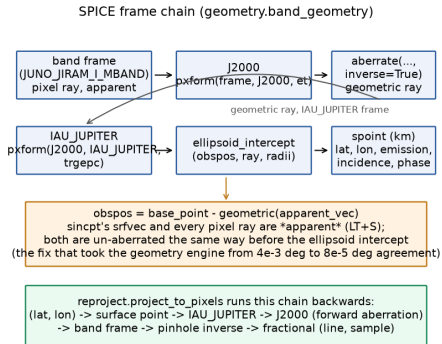


(e) The trackability heatmap



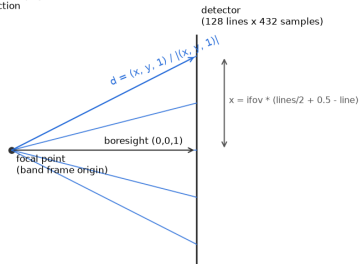
(f) The SPICE frame chain and the pinhole model

Figure (f): the SPICE engine's forward chain, and its inverse in reproject.py



pixel_directions: pinhole model in the band frame

y (samples) points out of the page;
x (lines) is the spin direction



Offline tests vs. gates

Offline tests

test*_offline.py

no mirror, no network

JIRAM_SKIP_GATES=1

synthetic fields, known answers

Gates

test_gate_*.py

need the real mirror + paper data

check against *published* ground truth

Gates are read-only to every executor, always. Passing a gate by editing it is task failure — stated in three separate project documents.

Where gate ground truth comes from

1. The 48-frame PJ4 fixture — numbers a peer-reviewed paper actually published
2. The published maps and TRACKER4 tables themselves, read directly with `vicar.py`
3. A *fresh*, independent SPICE oracle call — the only kind that can catch a bug in the *vectorisation itself*

Wrong gate #1: the geo consistency report

All-orbit residuals folded together well-conditioned frames and *partial* (limb) frames, where the label's "centre" is a visible-area centroid, not a boresight intercept.

"geo gate rewritten by lead (well-conditioned median 0.0078 deg; all-orbit report median was inflated by partial/limb frames)"

—build log, step 8a'

Fix: restrict to fully-on-planet, low-emission frames before computing the statistic.

Wrong gate #2: the best-baseline definition

Original `best_dt_s` rule (“closest to 5 px displacement”): 14/15 gate checks passed on first run — one failure, from PJ4’s dense polar campaign letting a far-in-time sequence coincidentally win. **Response: investigate, don’t loosen the gate.**

Redefined to “smallest dt among trackable partners” — no equivalent failure mode; gate tolerance widened to reflect real cadence spread (90 s), not to hide the bug.

The general rule

When a gate fails

Investigate why. Never relax the threshold or edit the gate to match whatever the code produced.

“If a threshold is not met, do not tune parameters to pass; investigate ... and report what limits agreement.”

—docs/specs/2026-09-05_tracking.md

Both wrong-gate stories: the *code* was never the bug — the gate’s own definition was measuring the wrong thing.

The loop, per milestone

1. **Facts first** — reconnaissance reports numbers, never designs
2. **Spec** — goal, files in scope, every decision explicit, exact validation block
3. **Gate before dispatch** — written by the lead, against ground truth
4. **Dispatch** — one executor, fresh context, the spec is the whole brief
5. **Verify** — scope check, gates run by the lead (not trusted from the report)
6. **Record** — outcome, executor, wall time, gate result → build log

Every spec ends: stop and list ambiguities rather than guess; list every judgment call.

What each executor tier actually did

- ▶ **Reconnaissance & prose:** lighter tier — 230k tokens settled the two facts the whole design rests on
- ▶ **Mechanical code, airtight spec:** cheapest executor that has passed a gate here — crawler + index, one pass
- ▶ **Bounded numerical/geometric work, SPICE/projection oracle, fresh-context review:** stronger tier — every milestone after the geometry engine, 15–45 min each, 1–3 genuine ambiguities each, resolved sensibly

The stalls, and the ambiguity stops that worked

The geometry engine, dispatched three times to the cheaper tier:

- ▶ Attempt 1 & 2: hit a genuine spec ambiguity, **stopped correctly** — exactly right
- ▶ Attempt 3: stalled **2h12m**, produced nothing — the failure mode with no automatic detector

Re-routed to the stronger tier: working engine, aberration fix included, in **16 minutes**.

Lesson: stopping on ambiguity $\neq$ failure. Silent non-progress needs a wall-clock budget, watched by whoever dispatched the work.

Errors executors caught, twice

- ▶ **Byte order** (little-endian, section 2.2): found *inside* the geometry-augmentation milestone by an executor trusting physical plausibility over the labels; the reprojection gate, written with the documented byte order, failed at the same moment and confirmed it independently
- ▶ **Stellar aberration** (section 4.4): a specific, measured $4\text{e-}3$ deg $\rightarrow$ $8\text{e-}5$ deg fix, found by an executor checking its own output against the same oracle the gate would later check

Neither was caught by a human re-reading the spec more carefully — both were caught by checking output against reality.

What this teaches about delegating numerical work

1. An executor that **stops on genuine ambiguity** is working correctly — never penalise this
2. A gate is only as good as its own definition of ground truth — distrust a failing gate's assumptions, not just the code under it
3. Genuine-oracle numerical work (a published number, an independent SPICE call, a known synthetic answer) is exactly where dispatch-then-verify-hard catches real errors — *in both directions*

Worked example: two extensions, chosen to contrast

1. A new named region: `great_red_spot` (a `local_ortho` patch)
2. A new statistic: skewness of the radiance field, $\langle (f - \mu)^3 \rangle / \sigma^3$

Same loop as every real milestone: facts $\rightarrow$ spec $\rightarrow$ gate $\rightarrow$ module $\rightarrow$ wiring $\rightarrow$ docs.

Step 1–2: facts, then the spec

Facts first: the storm's approximate centre lat/lon at a reference epoch; which orbits actually cover it. For skewness: run it once by hand on a real strip, check the existing `_prepare` masking helper is sufficient. **Spec:** files in scope, a read-only list, the exact API (`skewness(field, valid=None)` -> float, matching `structure_function`'s own calling convention), the validation block.

Step 3: the gate, written first

Region: no published storm ground truth exists — gate is *structural*: registry loads, round-trips to 1e-6 px, real-mirror selection is non-empty. Weaker than a scientific gate, and the spec should say so.

Statistic: a *real* numerical gate — synthetic field with known skewness, tight tolerance, plus a symmetric-field negative control at zero.

Step 4–6: module, wiring, docs

- ▶ Region: **4-line YAML addition** to `configs/regions.yaml` — no Python at all
- ▶ Statistic: `skewness` added next to `structure_function`, reusing `_prepare`; one new scalar attr on `strip_statistics`'s output
- ▶ Wiring: region needs none (`RegionGrid.from_yaml` is a generic lookup); statistic needs none beyond its own module (already owns its subcommand)
- ▶ Docs: one line each in `data_products.md`, `architecture.md`, `open_items.md` (no ground truth yet)

What the asymmetry teaches

- ▶ Region: almost no code, because `regions.py` was designed from the start to generalise past its initial four entries
- ▶ Statistic: real code, but slotted directly into an existing, well-established pattern — no new architecture needed

Neither extension needed a new pattern invented on the spot — that is itself a finding about the codebase, not just a tutorial.

Glossary: SPICE

- ▶ **SPICE**: NAIF's toolkit for spacecraft geometry from loaded *kernels*
- ▶ **Kernel** (SPK/CK/FK/IK/SCLK/LSK/PCK): trajectory / attitude / frames / instrument / clock / leap-seconds / planetary-constants files
- ▶ **Furnsh** / **kernel pool**: SPICE state is global per process — why every parallel worker here uses `spawn`, never `fork`
- ▶ **Ephemeris time (ET)**: SPICE's internal continuous time scale; this project's epoch is always the label's `START_TIME`, exactly
- ▶ **Sincpt** / **pxform**: surface-intercept and frame-rotation calls; `geometry.py` vectorises the same computation over a whole pixel grid
- ▶ **Light time** / **stellar aberration ("LT+S")**: finite light speed, and the observer's own velocity bending apparent direction — getting the *inverse* applied consistently is section 4.4's central subtlety

Glossary: geometry conventions

- ▶ **Planetocentric** latitude: angle from the equatorial plane to the vector from the body's *centre* — what this project uses everywhere
- ▶ **Planetographic** latitude: uses the local surface normal instead — numerically different on an oblate body, never used here
- ▶ **East-positive longitude, [0,360)**: this project's fixed convention, matching the archive's own labels
- ▶ **System III**: Jupiter's body-fixed rotating longitude system; IAU_JUPITER *is* System III for this project's purposes
- ▶ **Emission / incidence / phase angle**: angle to observer / to Sun / between the two, at the surface point
- ▶ **Orthographic / stereographic / equidistant / Lambert**: four radial laws relating map distance to true angular distance; the published maps use orthographic, despite the literature calling them stereographic

Glossary: archive & this codebase

- ▶ **PDS3 / PDS4**: the archive's label syntax (PDS3 text) and bundle layout (PDS4 directories) — not the same axis
- ▶ **RDR / EDR**: calibrated vs. raw data record — this project reads RDR exclusively
- ▶ **VICAR**: JPL's older self-contained image format — used here only because the ground-truth maps and vector tables ship in it
- ▶ **NCC**: normalised cross-correlation, 1.0 for identical patches — the tracker's match score *and* the general “does our reprojection match the paper” figure of merit
- ▶ **geo_ok / on_planet_frac / bore_***: SPICE succeeded at all / fraction of pixels on the ellipsoid / evaluated at the boresight specifically — consistent naming across every table in this codebase

Where to read next

full detail	PEDAGOGICAL_REVIEW.md
module-by-module facts	docs/architecture.md, docs/decisions.md
schemas	docs/data_products.md
command reference	docs/usage.md
open questions	docs/open_items.md

This deck and its figures: docs/pedagogy/